Travlr Getaways

CS 465 Project Software Design Document

Version 3.0

Table of Contents

Document Revision History

Version	Date	Author	Comments
1.0	05/16/26	Hassan Lindsay	Completed Milestone One sections: Executive Summary, Design Constraints and System Architecture View: Component Diagram
2.0	06/07/26	Hassan Lindsay	Completed Milestone Two Sections: Sequence Diagram, Class Diagram, API Endpoints Table and Supporting Architecture Documentation
3.0	06/10/26	Hassan Lindsay	Completed User Interface section, Angular SPA documentation. Testing procedures and Authentication Documentation

Executive Summary

Travlr Getaways requires a full stack travel booking application that supports a public customer-facing website, persistent trip and reservation data, and a secure administrator interface. The proposed architecture uses the MEAN stack: MongoDB for data storage, Express and Node.js for the server-side web application and RESTful services, and Angular for the administrator single-page application (SPA). This architecture separates public customer features from administrative management features while allowing both sides of the application to work with the same centralized data source.

The customer-facing side of the application will be delivered through an Express application using MVC routing, controllers, views, and Handlebars templates. Customers will be able to view travel packages, search for packages by location and price point, create accounts, book reservations, and later review their itineraries before travel. The customer website will begin with server-rendered pages and will progressively use dynamic data rather than static HTML.

The administrator side will be developed as an Angular SPA that communicates with the Express and Node.js back end through RESTful API endpoints. This admin SPA will allow authorized Travlr Getaways staff to maintain customer records, trip packages, pricing, and other travel data. Security will be added through authentication controls and protected endpoints so that administrative features are available only to approved users. This design gives Travlr Getaways a scalable foundation that can grow from a basic public site into a complete travel booking platform.

Design Constraints

The Travlr Getaways application has several design constraints that guide development. First, the application must be web based and accessible through standard browsers, which means the user interface must rely on HTML, CSS, JavaScript, and responsive web design practices. This constraint requires careful separation between browser-based presentation logic and server-side business logic so the application remains maintainable and can support different users and devices.

Second, the project must use the MEAN stack architecture. MongoDB stores travel package, customer, itinerary, reservation, and pricing data as NoSQL documents. Express and Node.js provide server-side routing, controllers, template rendering, and API services. Angular provides the rich administrative SPA used by Travlr Getaways staff. This constraint affects development because the application must be organized into clear layers, including views, routes, controllers, data models, API endpoints, and database schemas.

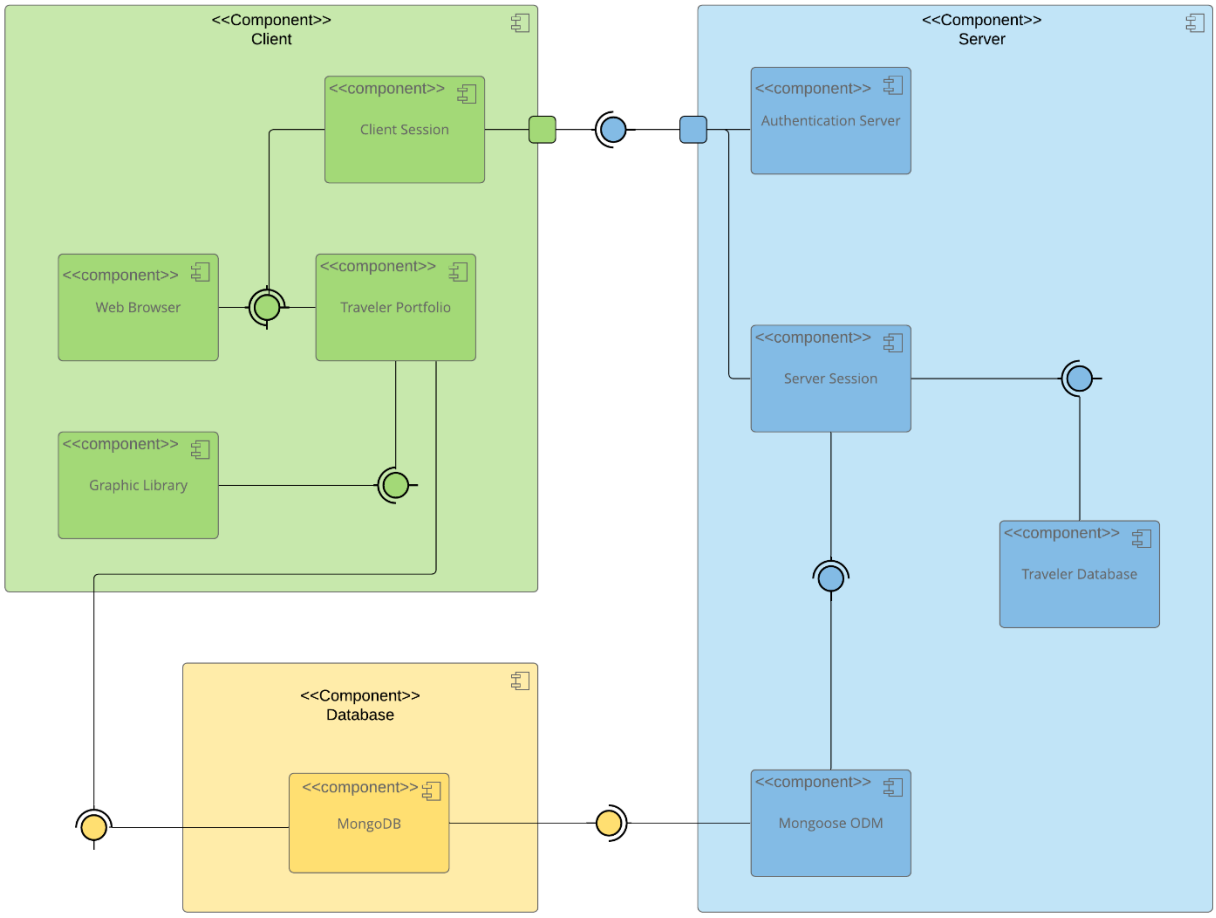
Third, the application must support both public customer functionality and restricted administrative functionality. This creates security and access-control constraints. Customer-facing pages must be available publicly, while administrative actions such as creating, editing, or deleting trip packages must require authentication and authorization. The implication is that the development process must include secure login behavior, protected API routes, and careful handling of user data.

Finally, the application must be built in stages while continuing to align with the client wireframe and software requirements. Static HTML pages are first converted into Express and Handlebars templates, then connected to JSON data, then connected to MongoDB through Mongoose, and eventually exposed through RESTful APIs and an Angular SPA. This staged approach creates a

constraint on development order, but it also reduces risk because each layer can be tested before the next layer is added.

System Architecture View

Component Diagram



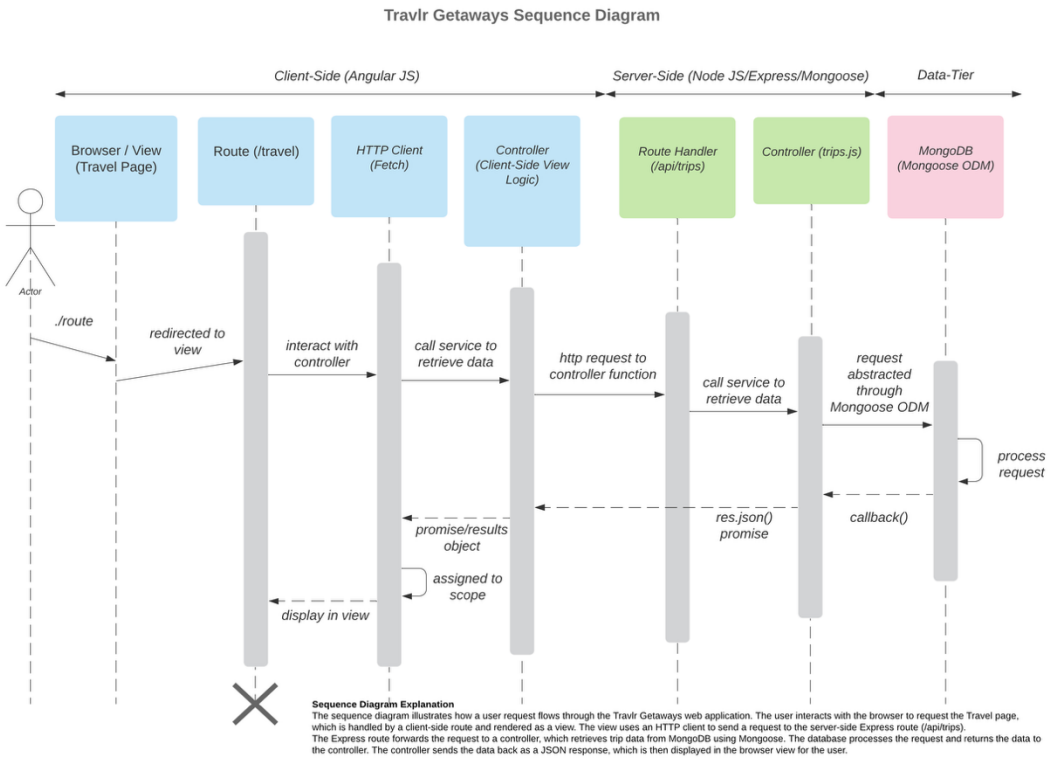
A text version of the component diagram is available: [CS 465 Full Stack Component Diagram Text Version].

The component diagram represents the Travlr Getaways application as a multi-tier full stack web system. At the presentation tier, users interact with the application through a web browser. Public customers use the customer-facing website to view travel packages, search for trip options, make reservations, and view itinerary information. Travlr Getaways administrators use an Angular SPA to manage application data through a richer client-side interface.

The application tier is built with Node.js and Express. Express handles incoming HTTP requests, routes those requests to the correct controller, renders Handlebars views for the public website, and exposes RESTful API endpoints for application data. The MVC structure separates responsibilities so that routes define URL entry points, controllers contain request-handling logic, views render the interface, and models represent the application data.

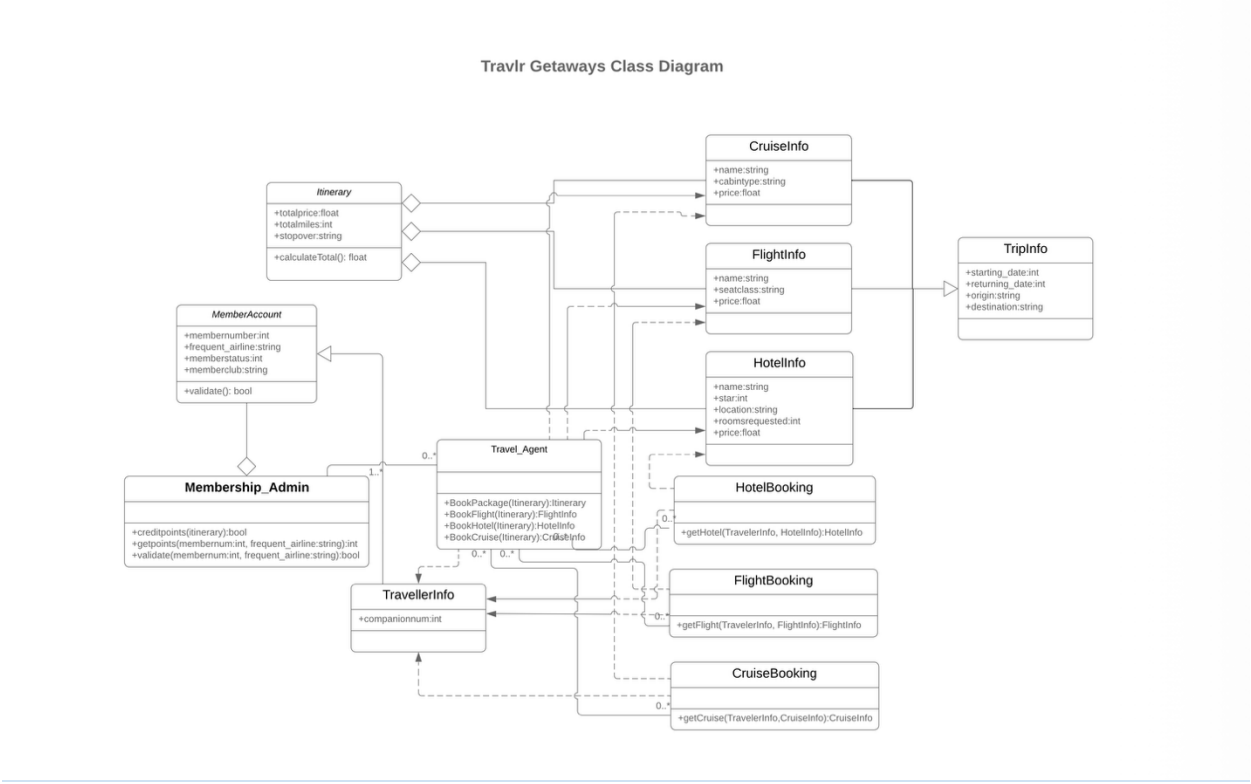
The data tier uses MongoDB as the NoSQL database, with Mongoose providing schemas and model access from the Express application. Trip packages, customers, pricing, itineraries, reservations, and administrative data are stored in MongoDB collections. The Express API communicates with MongoDB through these models, then returns data to either the customer-facing pages or the Angular admin SPA. This relationship allows the public website and administrator interface to share consistent data while maintaining clear separation between the user interface, server logic, and database storage.

Sequence Diagram



The sequence diagram shows how data moves through the Travlr Gateways full stack application. A user begins by requesting a page trough the browser. The browser request is handled by the client-side route and view logic. When the trip data is needed, the client-side controller uses the HTTP client to send a request to the Express API route. The Express route forwards the request to the trips controller, which communicates with the Mongoose model to retrieve trip data from MongoDB. MongoDB returns the requested documents, the controller sends the data back as a JSON response, and the browser renders the trip information in the view. This flow shows the separation between the presentation side, application side and data side.

Class Diagram

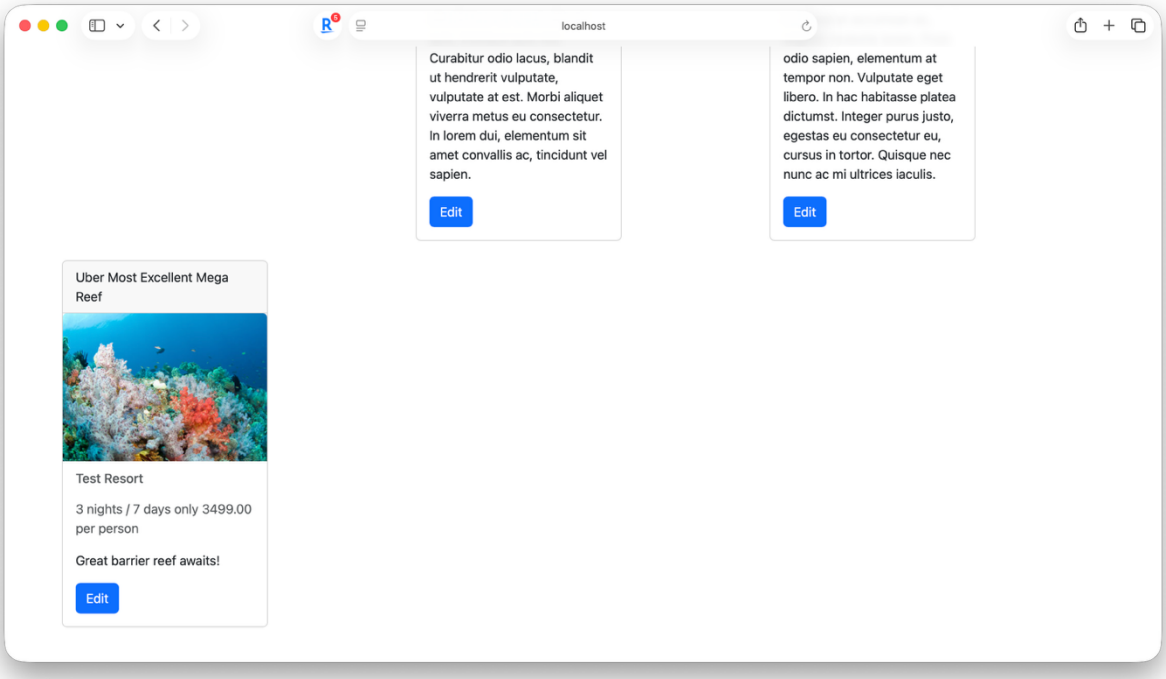


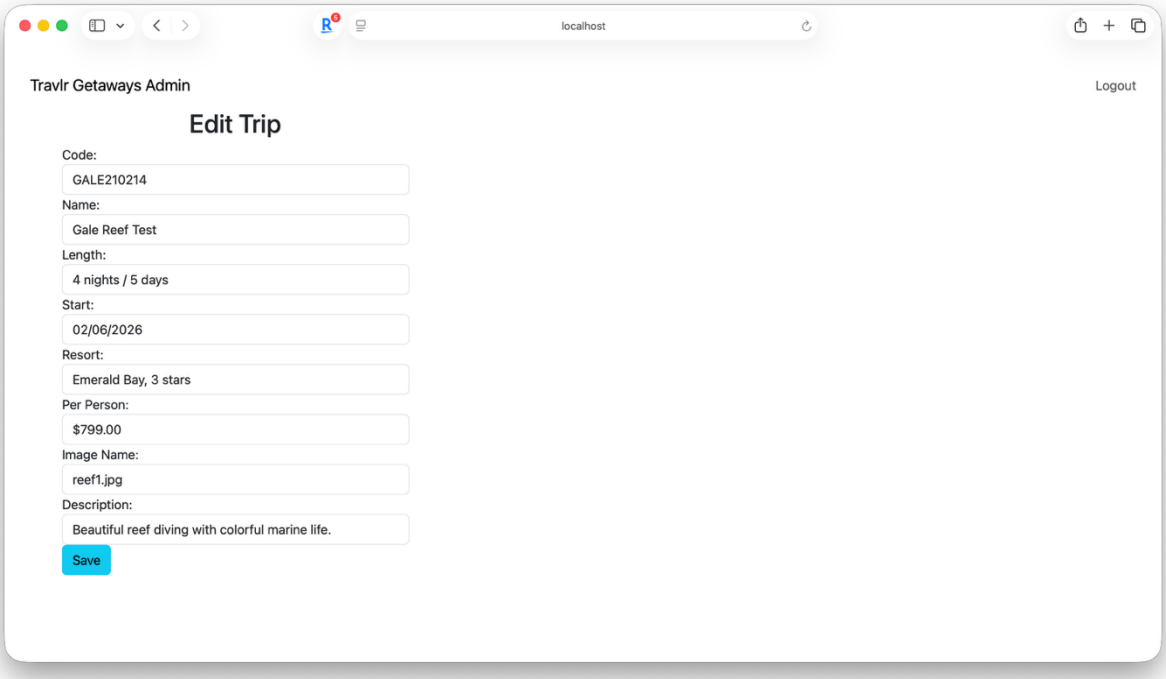
The class diagram represents the major JavaScript classes used by the Travlr Getaways application. The Itinerary class organizes the overall trip package and calculates the total cost. TripInfo stores the basic trip details, including the starting date, returning data, origin, and destination. CruiseInfo, FLightInfo and HotelInfo represent the travel service options that can be included in a trip. TravellerInfo stores traveler specific details. The Travel_Agent class coordinates booking actions by working with the itenarary and travel services classes. FlightBooking, HotelBooking and CruiseBooking manage the booking process for each service type. MemberAccount and Membership_Admin support member validation, rewards and account related functionality.

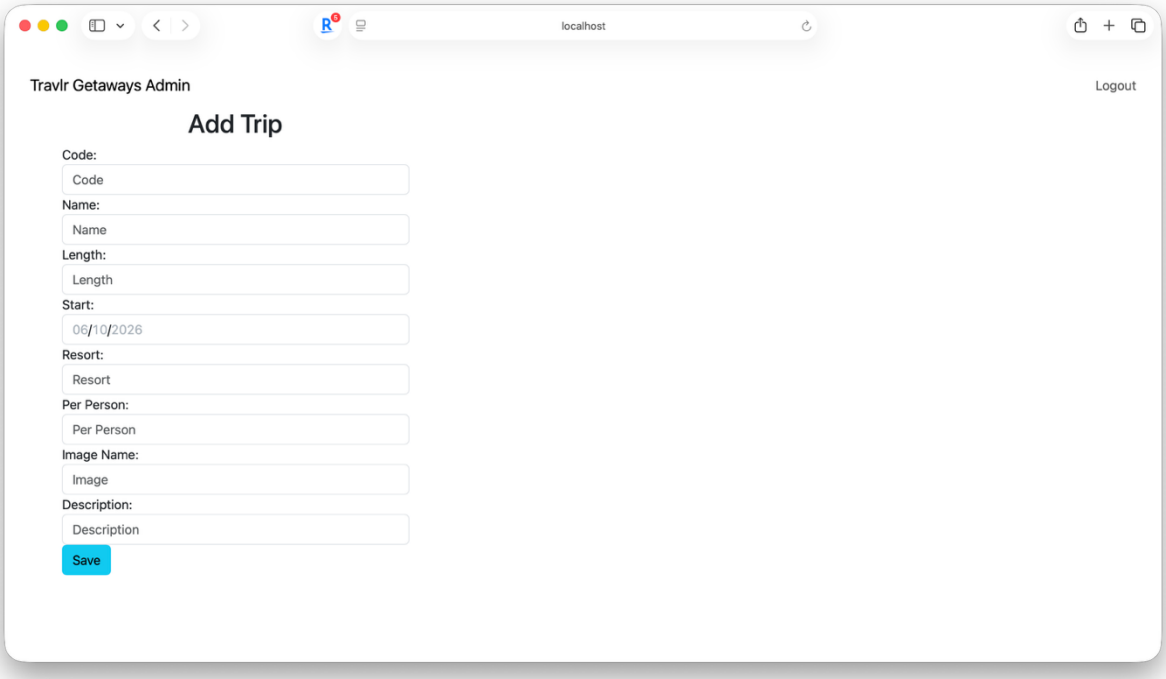
API Endpoints

Method	Purpose	URL	Notes
GET	Retrieve list of things	</api/trips>	Returns all trip packages from MongoDB as JSON
GET	Retrieve single thing	</api/trips/:tripCode>	Returns one trip based on the trip code in the URL
POST	Add a new trip	</api/trips>	Creates a new trip document in MongoDB
PUT	Update an existing trip	</api/trips/:tripcode>	Updates an existing trip based on the trip code

The User Interface


localhost
Curabitur odio lacus, blandit ut hendrerit vulputate, vulputate at est. Morbi aliquet viverra metus eu consectetur. In lorem dui, elementum sit amet convallis ac, tincidunt vel sapien.
Edit
odio sapien, elementum at tempor non. Vulputate eget libero. In hac habitasse platea dictumst. Integer purus justo, egestas eu consectetur eu, cursus in tortor. Quisque nec nunc ac mi ultrices iaculis.
Edit
Uber Most Excellent Mega Reef
Test Resort
3 nights / 7 days only 3499.00 per person
Great barrier reef awaits!
Edit


localhost
Travlr Getaways Admin
Logout
Edit Trip
Code:
GALE210214
Name:
Gale Reef Test
Length:
4 nights / 5 days
Start:
02/06/2026
Resort:
Emerald Bay, 3 stars
Per Person:
$799.00
Image Name:
reef1.jpg
Description:
Beautiful reef diving with colorful marine life.
Save



Angular Project Structure

The Angular administrator application is organized into components, services, models, and routes. Components such as Trip Listing, Add Trip, Edit Trip, Login, Navbar, and Trip Card provide the user interface and user interaction logic. Services such as TripDataService and AuthenticationService manage communication with the RESTful API and handle authentication functions. Models provide TypeScript definitions for application data including Trip, User, and AuthResponse objects. Angular routing is used to navigate between views without reloading the page.

The Angular project structure differs from the Express application structure. The Express application follows an MVC architecture consisting of routes, controllers, views, and models. Express is responsible for server-side processing, API endpoints, database access, and rendering the customer-facing website. Angular focuses on client-side functionality and user interaction while consuming data from the RESTful API.

Single Page Application Functionality

The Angular administrator site provides rich SPA functionality by allowing users to navigate between views without full page reloads. Administrators can view trip packages, add new trips, edit existing trips, and authenticate through the login screen. Angular components update dynamically when data changes, creating a more responsive user experience than a traditional multi-page application. The SPA communicates with the Express API through HTTP requests and displays updated information immediately after changes are made.

Testing GET and PUT Operations

Testing was performed throughout development to verify communication between the Angular SPA, Express API, and MongoDB database.

Initial API testing was completed using Postman. GET requests were sent to the /api/trips and /api/trips/:tripCode endpoints to verify that trip data could be successfully retrieved from MongoDB and returned as JSON responses. POST and PUT requests were also tested in Postman to confirm that new trip records could be created and existing trip records could be updated correctly in the database.

Database verification was performed using MongoDB Compass. After creating and updating trip records through the API, the trips collection was inspected directly within the MongoDB Compass to verify that the documents were stored correctly, and the updates were reflected in the database. This provided confirmation the Express API and Mongoose models were communicating successfully with MongoDB.

After the API endpoints were verified in Postman, testing was performed within the Angular SPA. The Trip Listing component was used to confirm that Angular successfully consumed data from the GET endpoints and displayed trip information correctly. Safari Browser developer tools and console logging were used to verify successful API responses.

PUT functionality was tested by selecting an existing trip through the Edit screen, modifying trip information, and submitting the changes. The Angular application sent a PUT request to the Express API, which updated the corresponding MongoDB document. The updated information was verified by refreshing the trip listing and confirming that the changes were reflected in the database.

Authentication testing was also completed by verifying successful login through the Angular login form, validating JWT token storage in localStorage, confirming conditional display of Login and Logout navigation options, and verifying that logout removed the authentication token and returned the application to an unauthenticated state. The generated JSON Web Token (JWT) was examined using JWT.io to validate its structure. This confirmed that the authentication system was generating valid JWT tokens and the Angular application was successfully storing and using the token for authenticated administrative access. Administrative API endpoints were protected using JWT authentication middleware to ensure only authenticated users could perform trip creation and update operations.